



Documentation Portfolio

Projet N°1

Détection des régimes de marché
avec Hidden Markov Models

Vincent GAUTHEREAU
2026

Table des matières

1. INTRODUCTION	4
2. Environnement technique	4
3. Données et prétraitement	5
3.1. Source des données	5
3.2. Calcul du ROC (Rate of Change)	5
4. Hidden Markov Model (HMM) : modèle statistique	5
4.1. Principe mathématique	5
4.2. Apprentissage	6
4.3. Génération des signaux	6
5. Backtesting	6
5.1. Backtest journalier	6
5.2. Backtest par trades	6
6. Gestion du risque.....	7
6.1. Sharpe Ratio	7
6.2. Volatilité	7
7. Simulation de Monte Carlo	7
7.1. Principe mathématique	7
7.2. Interprétation des résultats	8
8. Visualisation.....	8
9. Description du programme.....	9
9.1. Fichier principal (main.py)	9
9.1.1. Importation des bibliothèques et des modules	9
9.1.2. Fonction principal « main() »	9
9.1.3. Récupération	10
9.1.4. Construction de la dataframe principal	10
9.1.5. Backtesting et évaluation des performances	10
9.1.6. Simulation et visualisation	11
9.1.7. Commencement du fichier.....	11
9.2. Modules du programme	11
9.2.1. Data_loader.py	11
9.2.2. Indicators.py	11
9.2.3. Hmm_model.py	12
9.2.4. Signals.py.....	12
9.2.5. Backtest_daily.py	13
9.2.6. Backtest_trades.py	14

9.2.7.	Metrics.py	14
9.2.8.	Monte_Carlo.py	14
9.2.9.	Plots.py	16
10.	CONCLUSION/PERSONNELLE.....	17
11.	Sources	17
11.1.	Github	17
11.2.	IA.....	18
11.3.	YouTube.....	18

1. INTRODUCTION

Qui suis-je ?

Je m'appelle Vincent GAUTHEREAU, actuellement en 4e année à l'école d'ingénieur ICAM Toulouse. Fortement intéressé par la finance quantitative et le développement Python, ce projet s'inscrit dans ma démarche personnelle d'apprentissage et d'approfondissement de mes compétences en programmation et finance.

Pourquoi ce projet ?

Les marchés financiers sont caractérisés par une forte incertitude, une volatilité variable et des changements fréquents de comportements. Ces fluctuations reflètent l'évolution des conditions macroéconomiques, des sentiments des investisseurs et des événements géopolitiques. Détecter automatiquement ces changements de régime pourrait améliorer la prise de décision en trading.

Pour mener à bien ce projet, je me suis appuyé sur divers projets open-source GitHub, des vidéos YouTube et l'assistance de l'intelligence artificielle pour accélérer le développement. Cette transparence est importante : ces outils sont des amplificateurs de connaissance, pas des substituts.

Le projet

Un même actif peut successivement traverser des phases haussières (bull market), baissières (bear market) ou neutres (sideways market). L'identification de ces régimes de marché est importante pour adapter les stratégies d'investissement et optimiser la gestion du risque.

L'objectif de ce projet est de :

- Détecter automatiquement les régimes via le modèle HMM
- Générer des décisions de trading basé sur les régimes
- Évaluer les performances via un backtest, une différence entre du dit « buy and hold » littéralement acheté et attendre et notre stratégie.
- Projeter des scénarios futurs via une simulation de Monte Carlo

2. Environnement technique

Le projet a été développé en Python, langage de référence en finance quantitative, nous permet d'avoir :

- Une richesse en bibliothèques scientifiques, (Numpy, Pandas, hmmlearn, Plotly)
- adoption massive dans l'industrie financière.

Bibliothèques utilisées :

- pandas : permet la manipulation de séries temporelles financières
- Numpy : calculs matriciels et opérations vectorisées
- Hmmllearn : implémentation du HMM
- Yfinance : récupération des données de marché en temps réel
- Plotly : visualisations interactives et tableaux de board

3. Données et prétraitement

3.1. Source des données

Les données proviennent de Yahoo Finance via la bibliothèque yfinance

Les actifs peuvent inclure :

- Forex : EUR/USD, GBP/USD, USD/JPY
- Crypto-monnaies : BTC-USD, ETH-USD
- indices boursiers : S&P500, CAC40
- actions : AAPL, TSLA, MSFT

3.2. Calcul du ROC (Rate of Change)

Le ROC mesure la variation relative du prix sur une période donnée:

$$ROC = \frac{\text{Prix actuel} - \text{Prix } n \text{ période avant}}{\text{prix } n \text{ période avant}} \times 100$$

Cela permet de normaliser les mouvements du prix, indépendamment de son niveau absolu. De plus il permet la comparaison entre différents actifs. Ce calcul servira de base au HMM pour détecter les régimes de marché. D'autres indicateurs peuvent être ajoutés (RSI, MACD, Bollinger Bands) pour améliorer la robustesse du modèle.

4. Hidden Markov Model (HMM) : modèle statistique

4.1. Principe mathématique

Le HMM est un modèle probabiliste qui relie :

- États cachés (régimes) : haussier, neutre, baissier
- Observation visible avec le ROC des prix

Il suppose que l'état futur dépend uniquement de l'état actuel (propriété de Markov).

4.2. Apprentissage

Le HMM apprend via l'algorithme Expectation-Maximisation (EM) :

- Les probabilités de transition entre régimes
- Les paramètres, le rendement moyen et la volatilité associé de chaque régime
- La séquence d'états la plus probable

Cela permet en finance que le modèle soit adaptatif qui détecte les changements de régimes sans règles arbitraires

4.3. Génération des signaux

Régime détecté	Signal
Haussier	BUY
Neutre	HOLD
Baissier	SELL

5. Backtesting

Le backtesting consiste à tester la stratégie sur des données historiques pour évaluer ses performances. Deux approches sont utilisées.

5.1. Backtest journalier

Capital mis à jour chaque jour avec prise en compte des frais de transaction.

$$Capital_{t+1} = capital_t \times (1 + Position_t \times Return_t - Fees)$$

La stratégie est comparée à une stratégie Buy & Hold (achat et conservation) qui conserve une position longue constante comme un investissement à long terme.

$$Capital_{BH_{t+1}} = capital_{BH_t} \times (1 + Return_t)$$

5.2. Backtest par trades

Ici on a une approche qui analyse les performances trade par trade, permettant d'identifier les périodes rentables et risquées. Il suit une logique simple :

- Entrée : signal BUY ou SELL
- Sortie : changement de signal ou signal opposé
- Calcul du PnL par Trade avec frais inclus

On peut ensuite calculer les données :

- Nombre de trades gagnants / perdants
- Win rate = ratio des trades gagnants sur tous les trades passés
- Profit Factor = les gains générés par rapport aux pertes totales

6. Gestion du risque

6.1. Sharpe Ratio

Le ratio de Sharpe est l'indicateur de référence pour évaluer la performance ajustée du risque. Il mesure le rendement excédentaire par unité de risque. Voici la formule suivie :

$$Sharpe = \frac{R_{Stratégie} - R_{sans\ risque}}{\sigma}$$

$R_{Stratégie}$: rendement moyen

$R_{sans\ risque}$: rendement d'un actif sans risque

σ : volatilité du portefeuille

Nous pouvons l'interpréter comme ceci :

- Sharpe < 0 : performance inférieure aux taux sans risque
- 0 < Sharpe < 1 : performance acceptable mais risquée
- 1 < Sharpe < 2 : bonne performance ajustée au risque
- Sharpe > 2 : excellente performance

6.2. Volatilité

La volatilité est calculée sur des fenêtres mobiles appelée « rolling windows » pour capturer l'évolution du risque dans le temps. Cela nous permet d'identifier des périodes de forte incertitude (crises, événements), ajustement dynamique de la taille des positions, évaluation de la stabilité du modèle dans différents régimes de volatilité.

7. Simulation de Monte Carlo

la simulation de Monte Carlo est une méthode stochastique permettant de projeter de multiples scénarios futurs d'évolution des prix. Elle aide à quantifier l'incertitude et les risques extrêmes.

7.1. Principe mathématique

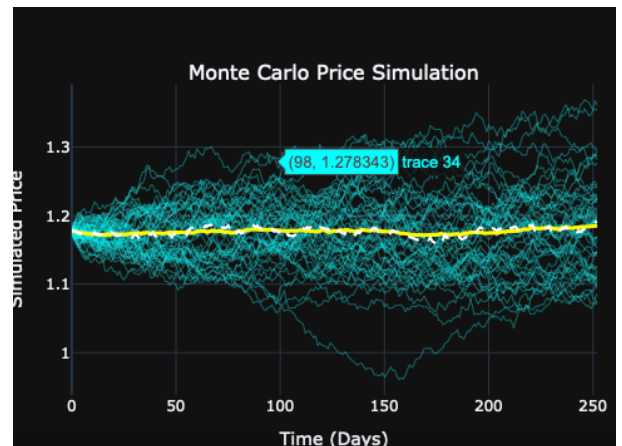
La simulation repose sur un processus stochastique, typiquement un Mouvement Brownien Géométrique, utilisé en finance pour modéliser l'évolution des prix

7.2. Interprétation des résultats

Coube moyenne (espérance) : Représente le scénario attendu basé sur les paramètres historiques.

Courbe médiane (scénario le plus probable) : Plus posé sur les valeurs extrêmes que la moyenne. En raison de la loi log-normale, médiane < moyenne.

Intervalle de confiance à 95% : Quantifie l'incertitude : 95% des trajectoires simulées se situent dans cet intervalle. La largeur de l'intervalle augmente avec l'horizon.



8. Visualisation

Les graphiques interactifs (via plotly) permettent une analyse visuelle complète de la stratégie. L'interactivité facilite l'exploitation des résultats.

On peut y retrouver :

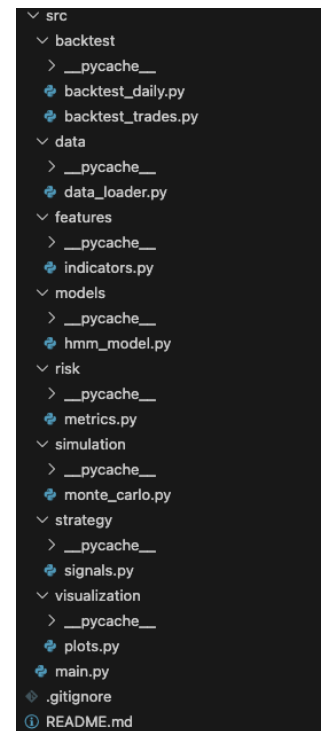
- Signaux de trading : visualisation des ordres passés
- Évolution du capital : comparaison stratégie vs Buy & Hold
- Volatilité historique 3D (temps, fenêtre, volatilité)
- Scénario de Monte Carlo : trajectoires simulées

9. Description du programme

Le programme est découpé en plusieurs sous fichier permettant d'avoir une meilleure maitrise des risques. De plus cette façon de fonctionner permet de faire de sorte d'améliorer le programme constamment avec une gestion des dossiers, et de pas surcharger le programme principal « main.py ».

Exemple si je veux ajouter un autre models a mon programme je peux ajouter une fichier python dans le dossier « models » comme un modèle SV : stochastiques volatility model et permet de l'appeler ou non dans le fichier principal comme une sorte de base de model déjà codé au paravent.

Voici en photo l'arborescence de mon algorithme. Cela me permet d'avoir un visuel beaucoup plus clair.



9.1. Fichier principal (main.py)

9.1.1. Importation des bibliothèques et des modules

Ce fichier principal nommé « main.py » permet de diriger l'entièreté du programme. Cela marche en appelant d'autre fichier. D'abord il faut les déclarer au début du programme :

```
from data.data_loader import fetch_data
from features.indicators import calculate_roc
from models.hmm_model import fit_hmm
from strategy.signals import generate_signals
from backtest.backtest_daily import run_daily_backtest
from backtest.backtest_trades import run_trade_backtest
from risk.metrics import sharpe_ratio
from simulation.monte_carlo import monte_carlo_simulation
from visualization.plots import plot_all
```

Ici par exemple pour avoir les datas, je vais appeler le fichier « data_loader » qui se trouve dans le dossier « data ». Et dans ce fichier data on veut la fonction « fetch_data ».

Aussi il faudra faire un import de bibliothèque, dans ce fichier celle qui nous intéresse est la bibliothèque « pandas », « `import pandas as pd` ». pandas est une bibliothèque Python utilisée pour manipuler et analyser des données sous forme de tableaux. Elle facilite le nettoyage, le calcul et la gestion des séries temporelles, notamment en finance.

9.1.2. Fonction principal « main() »

Ensuite nous pouvons faire la fonction « def main() », tout d'abord on configure les paramètres globaux :

```
def main():
    initial_capital = 10_000
    fee_rate = 0.001
    symbol = input("Choisir le symbol (ex: BTC-USD, EURUSD=X) : ")
```

9.1.3. Récupération

Les données de marché sont ensuite récupérées depuis la fonction `fetch_data()` du fichier `data_loader.py`, et on met comme paramètre le symbole dont on veut la data: `data=fetch_data(symbol)`

Ensuite on fait fonctionner l'indicateur ROC avec la fonction `calculate_roc()` dont on va mettre en paramètre les données « `data` » et nettoyé dont ce dont on a besoin.

```
roc = calculate_roc(data).dropna()
```

Puis nous allons utiliser cet indicateur pour entrainer le modèle HMM afin d'identifier les régimes de marché : `states = fit_hmm(roc)`

Ces régimes sont ensuite transformés en signaux de trading grâce à la fonction `generate_signals()` du fichier `signals.py` : `signals = generate_signals(roc, states)`

9.1.4. Construction de la dataframe principal

Nous créons une DataFrame pandas pour centraliser toutes les informations, les prix de clôture, les données de l'indicateur ROC, les régimes détectés, les signaux.

```
df = pd.DataFrame(index=roc.index)
df['Close'] = data.loc[roc.index, 'Close']
df['ROC'] = roc
df['Regime'] = states
df['Signal'] = signals
```

9.1.5. Backtesting et évaluation des performances

On veut exécuter le backtesting journalier dont on prendra en parametre le tableau avec les valeurs regrouper « `df` », le capital initial et les frais appliqués dans la fonction `run_daily_backtest()`, puis le backtest par trades avec les mêmes paramètres dans la fonction `run_trade_backtest()`

```
df = run_daily_backtest(df, initial_capital, fee_rate)
```

```
df_trades = run_trade_backtest(df, initial_capital, fee_rate)
```

Les performances sont ensuite évaluées à l'aide du ratio de Sharpe grâce à la fonction `sharpe_ratio()`, dont les résultats sont affichés dans la console pour comparer la stratégie au Buy & Hold.

```
sharpe_strat = sharpe_ratio(df['Strategy_Returns'].dropna())
sharpe_bh = sharpe_ratio(df['Returns'].dropna())
```

9.1.6. Simulation et visualisation

On veut maintenant faire une simulation de Monte Carlo réalisé à partir des prix de clôtures, donc on met en variable `mc_median` (la médiane des courbes de la simulation de Monte Carlo, puis, on visualise les résultats grâce aux graphiques grace à la fonction `plot_all()`.

```
mc_paths, mc_mean, mc_median = monte_carlo_simulation(df['Close'])
```

```
plot_all(df, df_trades, mc_paths, mc_mean, mc_median, symbol)
```

9.1.7. Commencement du fichier

On fait une condition qui permet de garantir que le programme s'exécute uniquement lorsque le fichier `main.py` est lancé directement.

```
if __name__ == "__main__":
    main()
```

9.2. Modules du programme

9.2.1. Data_loader.py

Le fichier `data_loader.py` contient la fonction `fetch_data()`, dont le rôle est de télécharger les données financières historiques d'un actif donné.

La fonction principale est de télécharger via `yfinance.download()` les données, de gérer les erreurs (symbole, vide...), de faire du nettoyage et de retourner les données demander par rapport au symbole dans la DataFrame pandas.

```
import yfinance as yf
from datetime import datetime, timedelta

def fetch_data(symbol, days=3650, interval='1d'):
    end = datetime.now()
    start = end - timedelta(days=days)
    return yf.download(symbol, start=start, end=end, interval=interval, auto_adjust=False)
```

9.2.2. Indicators.py

Le fichier contient les indicateurs techniques utilisés comme variables d'entrée pour les modèles. Ici nous avons choisi le ROC (Rate of Change). Le ROC mesure la variation relative du prix sur une période donnée.

```
def calculate_roc(data, window=12):
    return data['Close'].pct_change(window)
```

La fonction `pct_change(window)` calcule le rendement entre le prix courant et celui observé des périodes auparavant. Cet indicateur permet de capter la dynamique de

tendance, identifier les phases haussières ou baissières, servir de variable pour la détection de régimes.

9.2.3. Hmm_model.py

Le fichier contient la logique liée à l'entraînement et l'utilisation du Hidden Markov Model. Ce modèle permet d'identifier des régimes de marché cachés à partir d'un indicateur financier, ici basé sur le Rate of Change (ROC) il peut être incrémenter avec d'autre indicateur telle que le RSI ou les moyenne mobile (MM20..).

Normalisation :

Avant d'entraîner le modèle, les données sont normalisées à l'aide de la classe `StandardScaler`. Cela nous permet de centrer les données autour de zéro et réduire leur variance à unitaire. La normalisation est essentielle car les modèles probabilistes comme le HMM sont sensibles à l'échelle des données. Elle améliore la stabilité numérique et la convergence du modèle.

```
def fit_hmm(roc, n_states=3):
    scaler = StandardScaler()
    roc_scaled = scaler.fit_transform(roc.values.reshape(-1, 1))
```

modèle :

Le modèle utilisé est un Gaussian HMM, ou chaque régime de marché est représenté par une distribution gaussienne. Le paramètre `n_components` correspond au nombre de régimes de marché supposés (par défaut trois).

```
model = hmm.GaussianHMM(
    n_components=n_states,
    covariance_type="full",
    n_iter=100,
    random_state=42
)
```

Entraînement et prédiction :

```
model.fit(roc_scaled)    states = model.predict(roc_scaled)
```

Cette étape permet d'associer à chaque date un régime de marché estimé par le HMM. Les régimes sont ensuite retournés sous forme de séries pandas, suivant temporellement l'indicateur ROC.

9.2.4. Signals.py

Le fichier est responsable de la transformation des régimes de marché détecté par le HMM en signaux de trading exploitables. Tout d'abord on **aligne et sécurise les données**,

```
roc, states = roc.align(states, join="inner", axis=0)
```

Cette étape garantit que les deux séries temporelles sont parfaitement alignées dans le temps, évitant toute incohérence. Puis il identifie les régimes haussiers et baissiers.

```
regime_means = roc.groupby(states).mean()

regime_means = regime_means.sort_values()
```

La moyenne du ROC est calculée pour chaque régime. Les régimes sont ensuite classés du plus baissier au plus haussier. Le régime avec la plus faible moyenne est considéré comme baissier et le régime avec la plus forte moyenne est considéré comme haussier.

Ensuite on génère les signaux, haussier = +1 (position d'achat), neutre = 0 et baissier = -1 (position vendeuse),

```
bear_regime = regime_means.index[0]
bull_regime = regime_means.index[-1]
signal = pd.Series(0, index=roc.index)
signal.loc[states == bull_regime] = 1
signal.loc[states == bear_regime] = -1
```

9.2.5. Backtest_daily.py

Ce fichier contient la fonction `run_daily_backtest()`, dont l'objectif est de simuler l'évolution quotidienne du portefeuille en fonction des signaux de trading.

La fonction prend en paramètres le DataFrame principal `df`, contenant les prix et les signaux, le capital initial et le taux de frais appliqué à chaque transaction

Tout d'abord, les rendements journaliers du marché sont calculés :

```
df['Returns'] = df['Close'].pct_change()
```

La fonction `pct_change()` de la bibliothèque `pandas` permet de calculer une variation relative entre deux valeurs relatives.

Ensuite la position prise par la stratégie est définie à partir du signal de la veille afin d'éviter tout biais d'anticipation :

```
df['Position'] = df['Signal'].shift(1).fillna(0)
```

Les signaux de trading sont décalés d'une période à l'aide de la fonction `shift(1)`. Cela permet d'éviter l'anticipation et de prendre que la valeur $t-1$ à la position prise à t

La fonction `fillna(0)` permet de remplacer les valeurs manquantes par zéro et par conséquent ne pas prendre de position.

Les changements de position sont ensuite identifiés afin de détecter les transactions, et les frais associés sont calculés :

```
df['Trade'] = df['Position'].diff().abs()
```

La fonction `diff()` calcule la variation entre deux positions consécutives lorsqu'il y a un changement de position détecter une transaction à lieu. La fonction `abs()` permet d'avoir la valeur absolue qui permet de compter chaque changement de position

comme un transaction. Cela nous permet d'intégrer un coût réaliste à chaque opération, représentant ainsi la commission prélevée par les plateformes de trading.

Les rendements sont ensuite calculés tout en tenant compte à la fois de la performance du marché et des frais de transaction :

```
df['Strategy_Returns'] = (df['Position'] * df['Returns']) - df['Fees']
```

Enfin, l'évolution du capital est simulée pour la stratégie et une stratégie Buy & Hold, utilisée comme référence

```
df['Equity_Strategy'] = initial_capital * (1 + df['Strategy_Returns']).cumprod()
df['Equity_BuyHold'] = initial_capital * (1 + df['Returns']).cumprod()
```

La fonction `cumprod()` permet de calculer le produit cumulé des rendements, ce qui permet de simuler l'évolution du capital dans le temps. Puis on renvoie notre valeur à la DataFrame enrichi, qui contiendra l'ensemble des résultats concernant le backtesting journalier.

9.2.6. Backtest_trades.py

Ce fichier contient la fonction `run_trade_backtest()` qui permet d'analyser les performances Trade par Trade, contrairement au backtesting journalier. Cette fonction a un peu la même structure que le fichier backtest journalier mais, là on va pouvoir avoir un calcul du P&L (Profit and Lose), indicateur de performance des trade, calcul aussi à chaque fois la valeur du portefeuille.

9.2.7. Metrics.py

Le fichier regroupe les indicateurs de performance utilisés pour évaluer la stratégie de trading. Dans notre cas on utilisera que le ratio de Sharpe. Le ratio de Sharpe mesure la performance d'une stratégie ajustée du risque. Il compare le rendement moyen obtenu à la volatilité des rendements.

```
import numpy as np

def sharpe_ratio(returns, periods=252):
    return np.sqrt(periods) * returns.mean() / returns.std()
```

La multiplication par la racine du nombre de périodes permet d'annualiser le ratio. Un ratio de Sharpe élevé indique une meilleure performance pour un niveau de risque donné.

9.2.8. Monte_Carlo.py

Le fichier permet une simulation de Monte Carlo, utilisée pour projeter différents scénarios stochastiques futurs d'évolution des prix.

Estimation des paramètres statistiques :

```
def monte_carlo_simulation(prices, n_simulations=50, n_days=252):
    last_price = prices.iloc[-1]
    returns = prices.pct_change().dropna()

    mu = returns.mean()
    sigma = returns.std()
```

Les rendements historiques sont utilisés pour estimer : la moyenne des rendements et leur volatilité.

Génération des scénarios :

Pour chaque simulation, un paths aléatoire est généré selon une loi normale

```
paths = []
for _ in range(n_simulations):
    path = [last_price]
    for _ in range(n_days):
        shock = np.random.normal(mu, sigma)
        path.append(path[-1] * (1 + shock))
    paths.append(path)
```

Qui est ensuite ajouter dans une liste

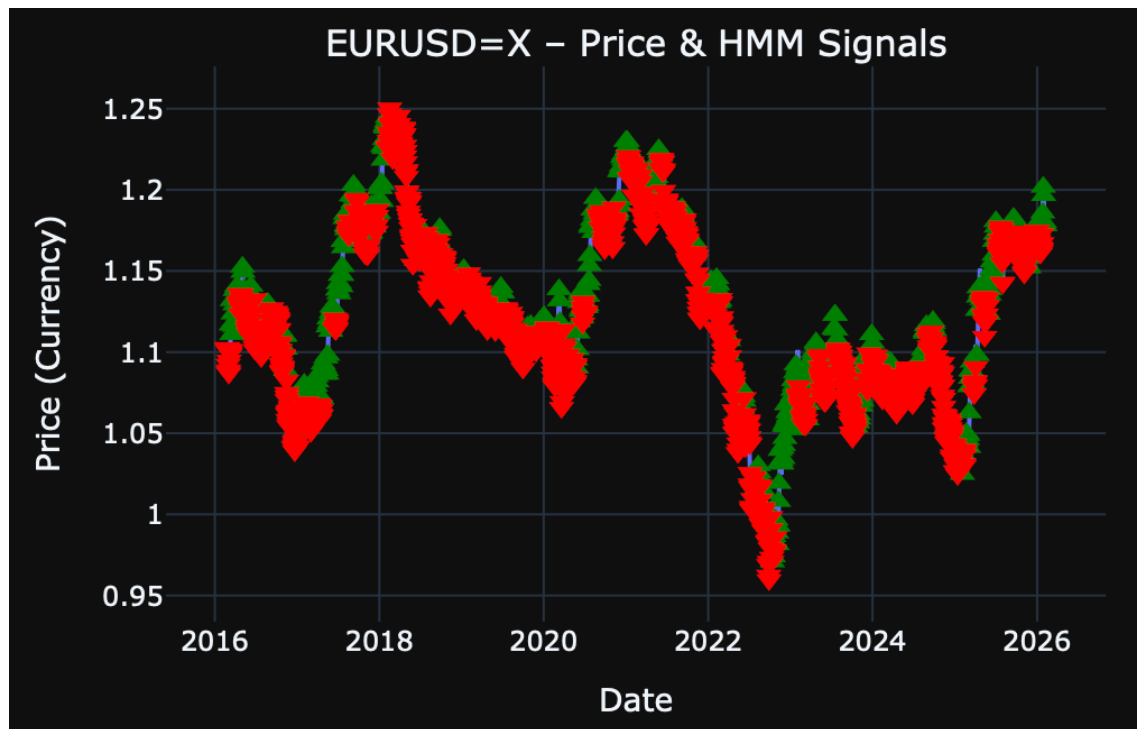
Résultats de la simulation :

La fonctions retourne l'ensemble des trajectoires simulées, la trajectoire moyenne et la trajectoire médiane. Ces éléments permettent d'analyser les scénarios probables et les risques extrêmes.

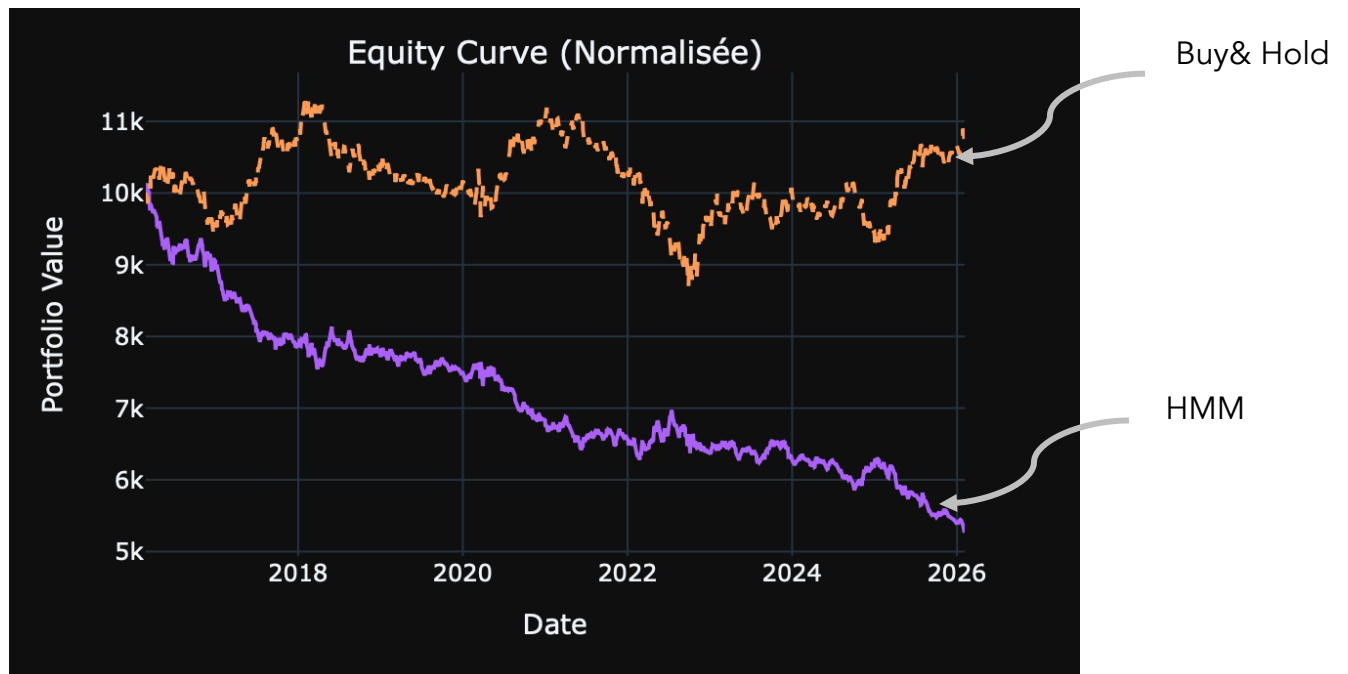
```
paths = np.array(paths)
return paths, paths.mean(axis=0), np.median(paths, axis=0)
```

9.2.9. Plots.py

Les graphiques interactifs (via Plotly) permettent de visualiser l'évolution des prix et des régimes détectés :

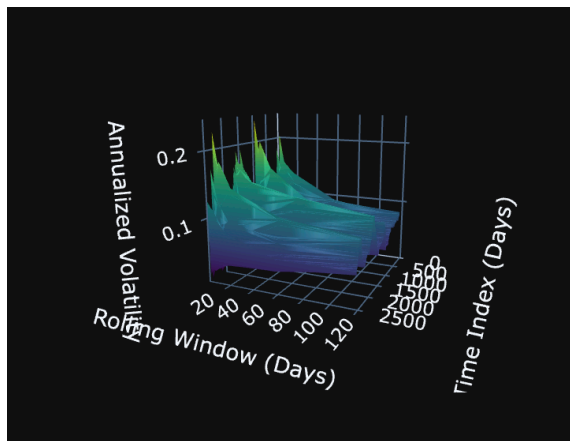


L'évolution du capital (stratégie vs Buy & Hold) :



Ici on peut voir que notre stratégie n'est pas bonne comparé à la stratégie Buy & Hold.

La volatilité historique en 3D



On est ici sur l'EURUSD, on voit une très forte volatilité qui est en corrélation avec le graphique précédent.

10. CONCLUSION/PERSONNELLE

Ce projet a permis de construire un système fonctionnel de détection de régimes avec le HMM. L'objectif n'était pas de créer une stratégie miracle, mais de développer un outil de détection de régimes comme aide à la décision.

Cela m'a permis de comprendre les fondements mathématiques des HMM, d'améliorer ma maîtrise du langage Python et de découvrir des bibliothèques performantes comme Plotly pour créer des Dashboard interactifs.

Cet algorithme a bien sûr des limites : utilisation d'un seul indicateur, absence de position sizing, hypothèses simplifiées pour Monte Carlo. Cependant, la structure modulaire de l'algorithme permet son amélioration facile.

Ce projet a été un excellent apprentissage et quelque chose de très enrichissant à concevoir. D'autres projets de finance quantitative sont à venir. Je suis ouvert aux retours et aux discussions !

11. Sources

11.1. Github

- <https://github.com/loelschlaeger/fHMM>
- <https://github.com/olafSmits/MonteCarloMethodsInFinance>
- <https://github.com/fyangch/crypto-dashboard>
- <https://github.com/vinsgte/HMM-Backtesting>

11.2. IA

- <https://claude.ai>
- <https://grok.com>

11.3. YouTube

- <https://www.youtube.com/watch?v=b9RgHa1CnH4>
- <https://www.youtube.com/watch?v=uOqPElBsBmM&list=PLPe-ytPHqyh3E5EdDAxOwHQkB0eDOPZb>
- <https://www.youtube.com/watch?v=9RKw86g4IFc>